

Panduan Dasar: Event-Driven Architecture (EDA) & Kafka Acks

Penjelasan sederhana untuk pemula

1. Apa itu Event-Driven Architecture (EDA)?

Secara sederhana, **EDA** adalah pola desain di mana komponen-komponen dalam sistem berkomunikasi dengan cara saling mengirim "Event" (peristiwa/perubahan data) secara *asynchronous* (tidak saling menunggu).

3 Komponen Utama EDA

- **1. Event Producer:** Layanan yang membuat atau memicu event.
- **2. Message Broker:** Perantara yang menerima event dari producer dan meneruskannya (Contoh: Apache Kafka, RabbitMQ).
- **3. Event Consumer:** Layanan yang "mendengarkan" event tertentu dan bereaksi terhadapnya.

2. Contoh Kasus: Sistem E-Commerce

Bayangkan Anda menekan tombol "Checkout" saat membeli barang.

 **Cara Lama (Synchronous / Monolith)**

Layanan Order memanggil Inventaris → memanggil Pembayaran → memanggil Email.

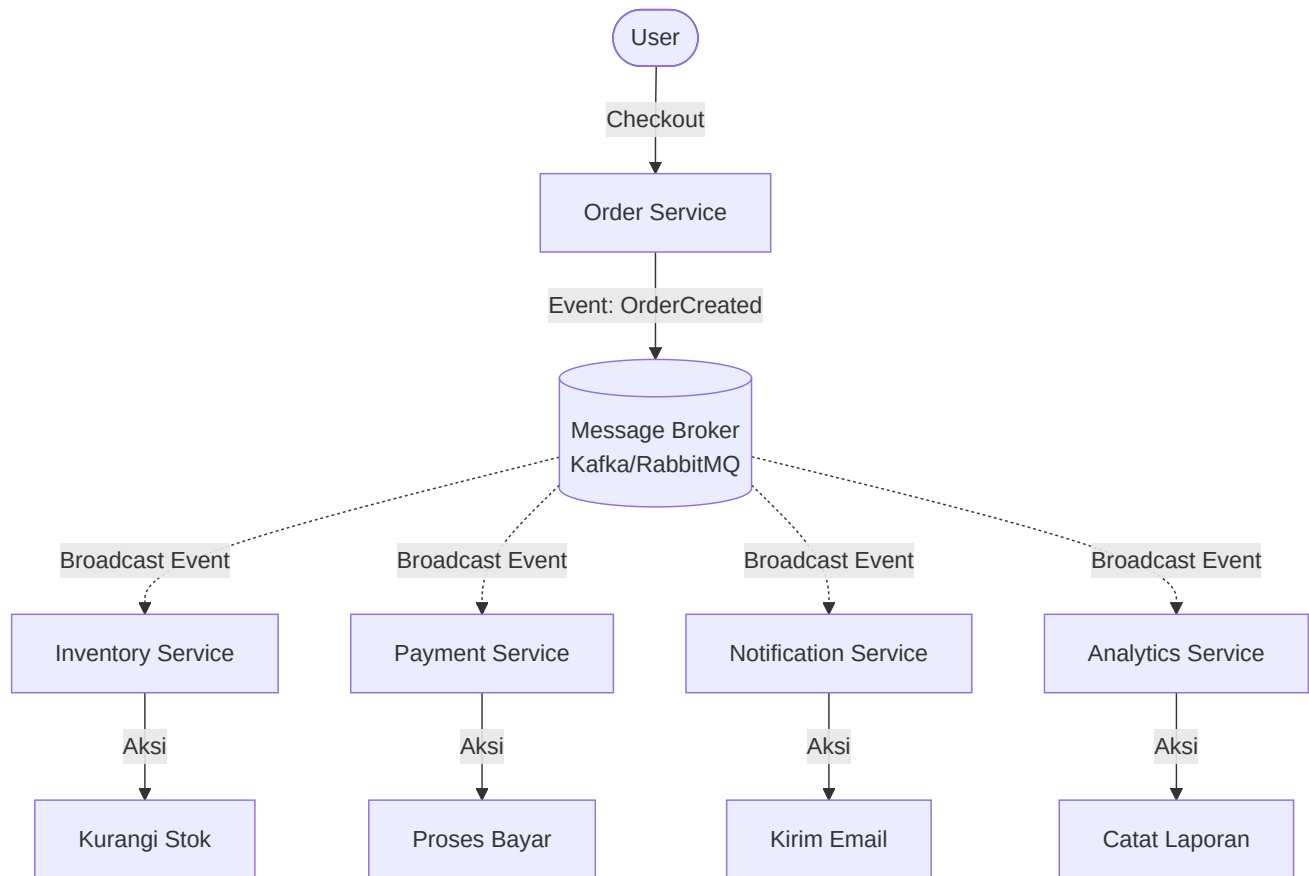
Masalah: Jika Layanan Email mati, proses checkout bisa gagal atau timeout karena sistem saling menunggu (blocking).

✓ Cara Event-Driven (Asynchronous)

Sistem tidak saling memanggil langsung, mereka hanya melempar pengumuman (Event).

1. User menekan Checkout.
2. **Order Service** membuat event `OrderCreated` ke Broker.
3. Layanan lain yang "mendengar" akan bereaksi sendiri-sendiri:
 - 📦 **Inventory** → Memotong stok.
 - 💳 **Payment** → Memproses tagihan.
 - ✉️ **Notification** → Mengirim email ke user.

🌐 Ilustrasi Alur EDA



3. Mengapa Menggunakan Event-Driven?

- 🔌 **Loose Coupling (Mandiri):** Layanan tidak saling bergantung. Menambah layanan baru sangat mudah tanpa mengubah kode layanan lama.
- 🛡️ **Resilience (Tahan Banting):** Jika satu layanan mati (misal Notifikasi), event tidak hilang tapi menumpuk di Broker dan akan diproses saat layanan hidup kembali.
- 📈 **Scalability (Mudah Skala):** Saat event melonjak (misal: promo Harbolnas), kita bisa menambah server untuk layanan tertentu saja agar proses lebih cepat.
- ⚡ **Real-time Processing:** Sangat responsif, cocok untuk deteksi fraud pada transaksi.

🔧 Teknologi Populer

4. Pengaturan 'Acks' pada Apache Kafka

Saat mengirim pesan ke Kafka, kita butuh jaminan apakah pesan tersebut sampai atau tidak. Ini diatur menggunakan parameter **acks** pada Producer (pengirim). Tidak ada nilai *true/false*, melainkan level pengakuan.

acks=0 (Tembak dan Lupakan)

- **Cara kerja:** Kirim pesan tanpa menunggu konfirmasi Broker.
- **Kelebihan:** Paling cepat.
- **Kekurangan:** Risiko data hilang sangat tinggi jika Broker mati.
- **Gunakan untuk:** Log klik user, metrik monitoring (data tidak kritis).

acks=1 (Standar / Leader Acknowledges)

- **Cara kerja:** Menunggu konfirmasi dari Server Utama (Leader) saja.
- **Kelebihan:** Performa baik dan cukup aman.
- **Kekurangan:** Data bisa hilang jika Leader mati sebelum replikasi ke server cadangan.
- **Gunakan untuk:** Feed media sosial, notifikasi standar.

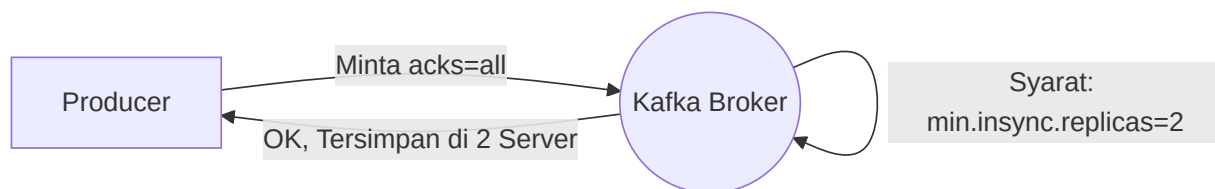
acks=all atau acks=-1 (Paling Aman / Full ISR)

- **Cara kerja:** Menunggu konfirmasi dari Leader DAN semua server cadangan (ISR).
- **Kelebihan:** Risiko data hilang nyaris 0 (Zero Data Loss).
- **Kekurangan:** Paling lambat.

- **Gunakan untuk:** Transaksi finansial, E-Commerce pesanan.

5. Hubungan acks dengan EDA

Dalam EDA, komponen saling lepas, jadi kita butuh **Delivery Guarantee** (Jaminan Pengiriman) agar event tidak hilang. acks adalah "katup pengaman" tersebut.



Siapa yang mengatur?

- **Producer** mengatur level acks (Seberapa kuat minta resi tanda terima).
- **Broker/Topic** mengatur aturan `min.insync.replicas` (Syarat minimal server cadangan yang harus aktif untuk menerima `acks=all`).

Jika Producer minta `acks=all` tapi server aktif kurang dari syarat minimum, Kafka akan menolak pesanan tersebut untuk menjaga keamanan data.